Web Framework and Services

- **Credit: 4**
- **Minimum weeks per Semester : 15 (Including class work, examination, preparation etc.)**
- **Purpose of Course :** To make students aware of Open Source Web Based Tools and Database
- **Course Objective**

1. To make students understand the concepts of Open Source Web Based Dynamic Scripting Language.
2. To make students understand the concepts of Open Source Database.

- **Pre-requisite :** Basic knowledge of Scripting Language & HTML and Python.
- **Course outcome :** Ability to develop Web Based Applications.

Dr. Jyoti Rana, Naran Lala College, Navsari

UNIT-1: PHP FUNDAMENTALS

1.1 Concepts of Php and introduction

1.2 Php syntax: variables, constants, echo and print commands

1.3 Data types

1.4 Operators, Conditional Statements (if. Else, Switch. Case), Arrays

1.5 Sorting Arrays, Php Loops

Dr. Jyoti Rana, Naran Lala College, Navsari

Client Side Scripting versus Server Side Scripting

BASIS FOR COMPARISON	CLIENT-SIDE SCRIPTING	SERVER-SIDE SCRIPTING
Basic	Works at the front end and script are visible among the users.	Works in the back end which could not be visible at the client end.
Processing	Does not need interaction with the server.	Requires server interaction.
Languages involved	HTML, CSS, JavaScript, etc.	PHP, ASP.net, Ruby on Rails, ColdFusion, Python, etcetera.
Affect	Can reduce the load to the server.	Could effectively customize the web pages and provide dynamic websites.
Security	Insecure	Relatively secure.

1.1.3 Installation and configuration of PHP and MYSQL

- Apache official website
  - www.apache.org
- PHP official website
  - www.php.net
- MYSQL official website
  - www.mysql.com
- To install PHP, we will suggest you to install AMP (Apache, MySQL, PHP) software stack. It is available for all operating systems. There are many AMP options available in the market that are given below:
  - **WAMP** for Windows
  - **LAMP** for Linux
  - **MAMP** for Mac
  - **SAMP** for Solaris
  - **FAMP** for FreeBSD
  - **XAMPP** (Cross, Apache, MySQL, PHP, Perl) for Cross Platform: It includes some other components too such as FileZilla, OpenSSL, Webalizer, Mercury Mail, etc.
  - If you are on Windows and don't want Perl and other features of XAMPP, you should go for WAMP. In a similar way, you may use LAMP for Linux and MAMP for Macintosh.

PHP

- PHP is an open-source, interpreted, and object-oriented scripting language that can be executed at the server-side. PHP is well suited for web development. Therefore, it is used to develop web applications (an application that executes on the server and generates the dynamic page.).
- PHP was created by **Rasmus Lerdorf in 1994**
- **Open source** means source code is available free to the public. So one can download it, use it and share it.

PHP

- PHP stands for Hypertext Preprocessor.
- PHP is an interpreted language, i.e., there is no need for compilation.
- PHP is faster than other scripting languages, for example, ASP and JSP.
- PHP is a server-side scripting language, which is used to manage the dynamic content of the website.
- PHP can be embedded into HTML.
- PHP is an object-oriented language.
- PHP is an open-source scripting language.
- PHP is simple and easy to learn language.
- PHP files have extension ".php"

Terms

Preprocessor: Preprocessor is a program that processes its input data to produce output that is used as input to other program.

As PHP script is executed on the server, and the plain HTML result is sent back to the browser.

Hypertext: it is a text displayed on a computer with reference to other text that reader can immediately access by mouse click or keypress. It contains text, images, audio, video etc.

Open Source: it is a computer software within which source code is made available with a license. Copyright holder provides right to

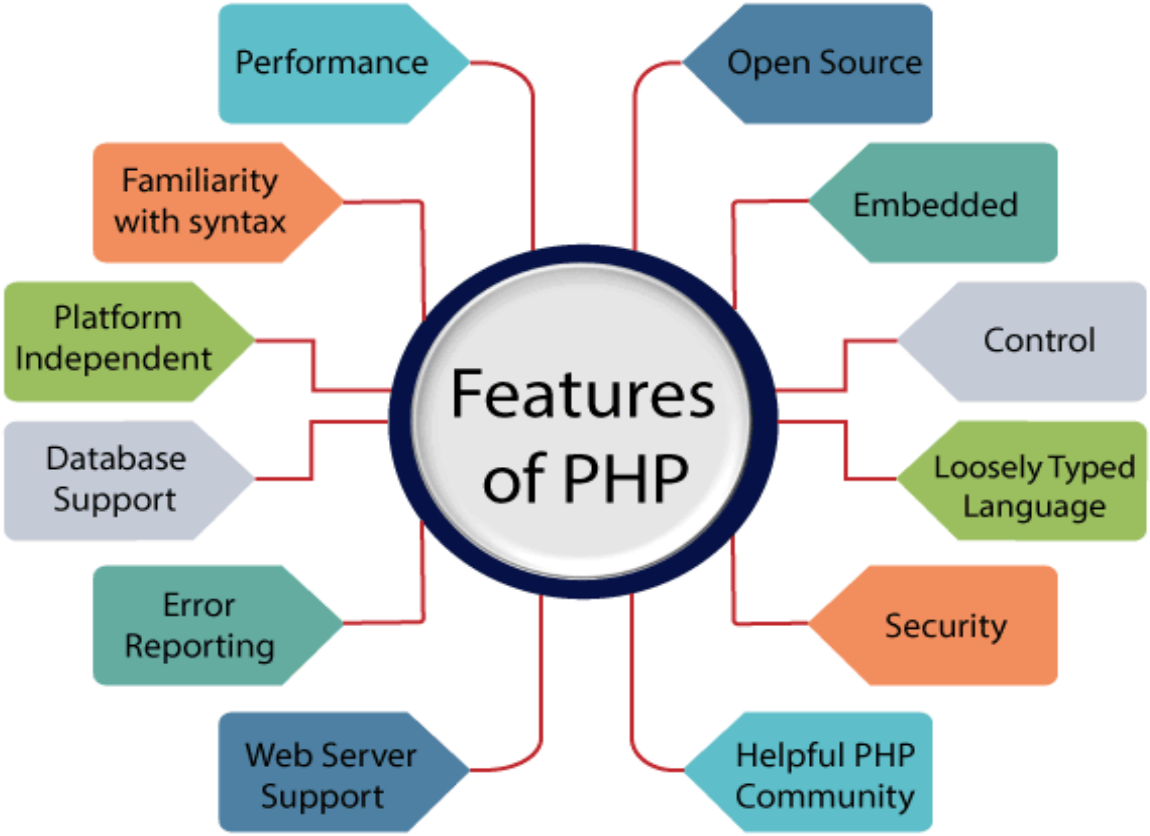
Why use PHP?

- PHP is a server-side scripting language, which is used to design the dynamic web applications with MySQL database.
- It handles dynamic content, database as well as session tracking for the website.
- You can create sessions in PHP.
- It can access cookies variable and also set cookies.
- It helps to encrypt the data and apply validation.
- PHP supports several protocols such as HTTP, POP3, SNMP, LDAP, IMAP, and many more.
- Using PHP language, you can control the user to access some pages of your website.
- As PHP is easy to install and set up, this is the main reason why PHP is the best language to learn.
- PHP can handle the forms, such as - collect the data from users using forms, save it into the database, and return useful information to the user. **For example** - Registration form.

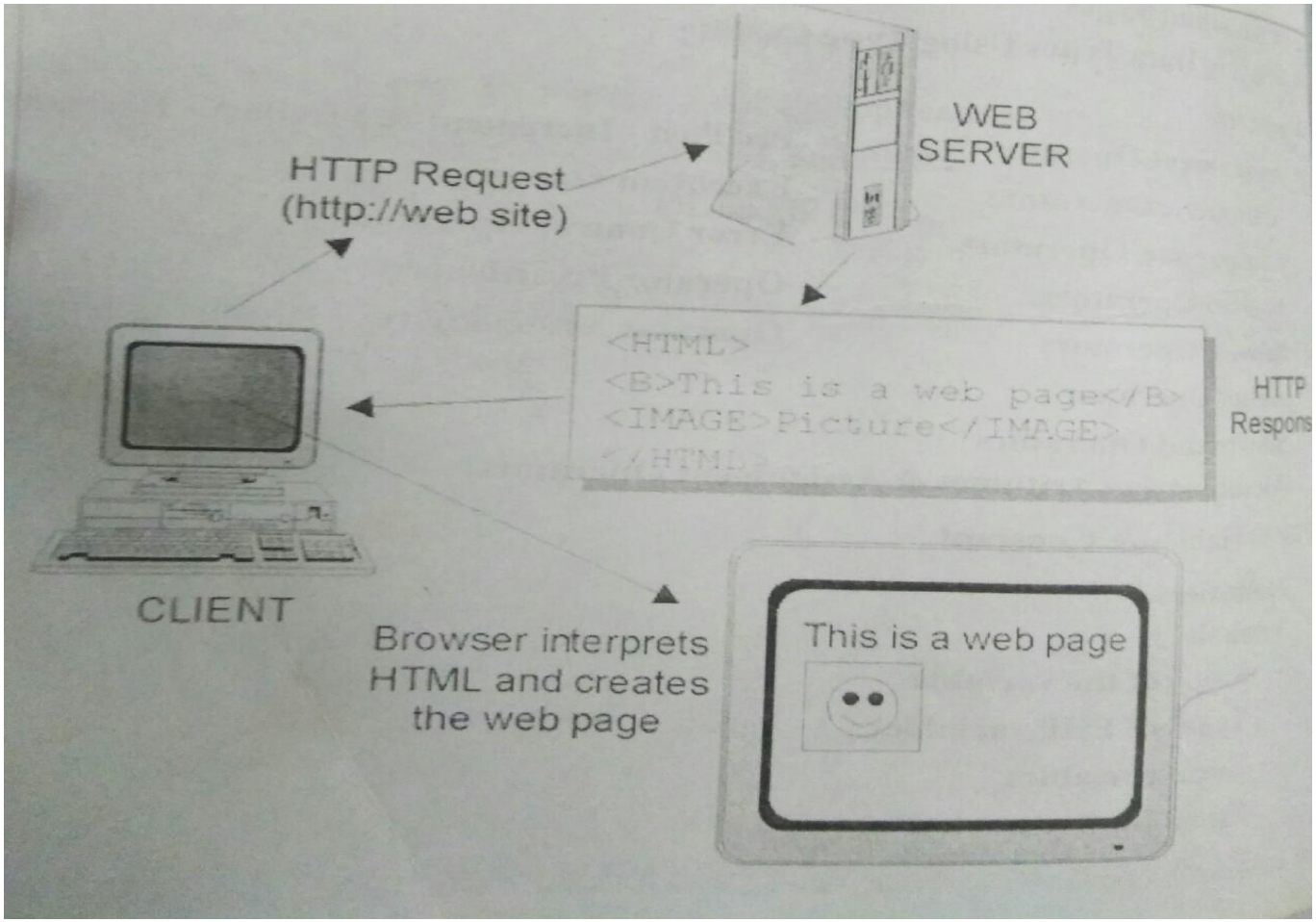
Frameworks for PHP Web Development

Dr. Jyoti Rana, Naran Lala College, Navsari

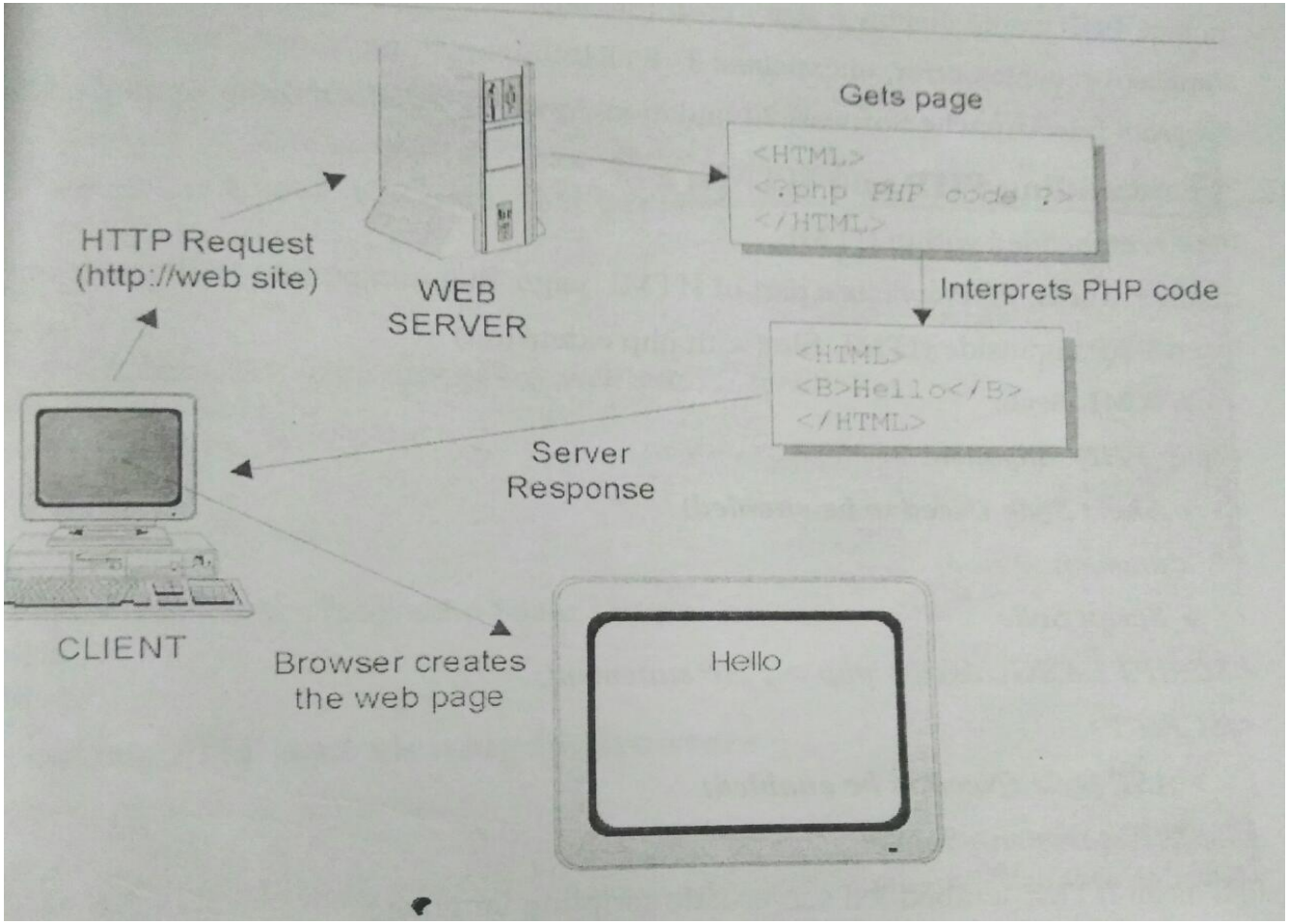
PHP Features



What Happens to HTML Page?



Parsing of PHP Code



Parsing of PHP Code

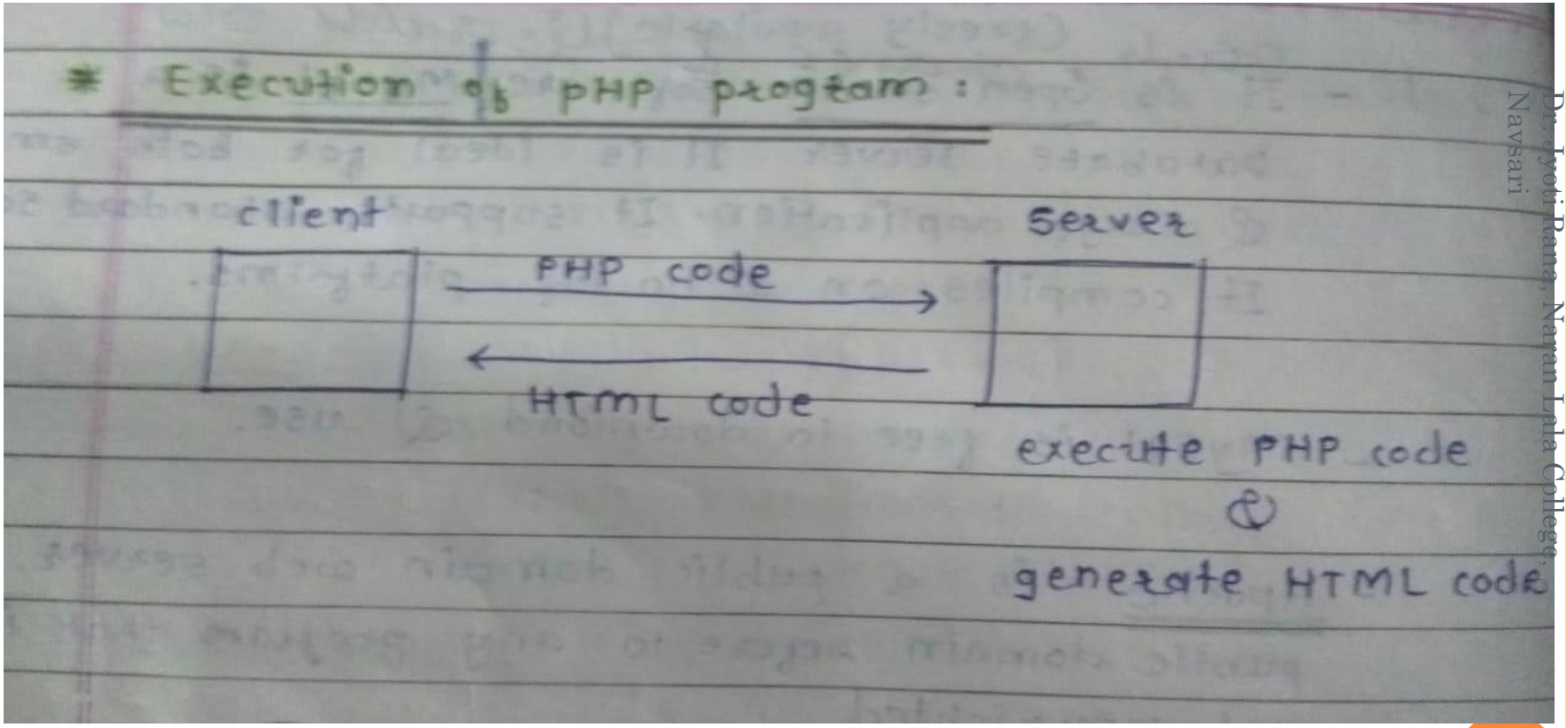
- Read the request from the browser
- Find page in the server
- Perform instructions to modify the page provided in PHP.
- Send the page back to the browser via Internet.

Parsing of PHP Code

- XML Style
  - <?php
    Php code
    ?>
- Short Style
  - <?
    Php code
    ?>
- Script Style
  - <script language="php">
    Php code
    </script>
- ASP style
  - <%
    Php code
    %>

Dr. Jyoti Rana, Naran Lala College, Navsari

Execution of PHP Program



Embedding PHP and HTML

- We can embed html into php as and when required and vice versa.
- Embedding PHP into HTML.
  -
```
<!DOCTYPE html>
<html>
<body>

<h1>My first PHP page</h1>

<?php
echo "Hello World!";
?>

</body>
</html>
```


Dr. Jyoti Rana, Naran Lala College, Navsari

Embedding HTML into PHP

```
<?php
   echo "<h1> Welcome Students!";
   echo "<h2> Have a great day";
?>
```

Output

Welcome Students!

Have a great day

Dr. Jyoti Rana, Naran Lala College, Navsari

1.2 PHP Syntax: Variables, Constants, echo and print commands

PHP Variables

- In PHP, a variable is declared using a **$ sign** followed by the variable name. Here, some important points to know about variables:
- As PHP is a loosely typed language, so we do not need to declare the data types of the variables. It automatically analyzes the values and makes conversions to its correct data type.
- After declaring a variable, it can be reused throughout the code.
- Assignment Operator (=) is used to assign the value to a variable.
- Syntax of declaring a variable in PHP is given below:
  - $variablename=value;

Dr. Jyoti Rana, Naran Lala College, Navsari

Rules for Declaring PHP Variables

- A variable must start with a dollar ($) sign, followed by the variable name.
- It can only contain alpha-numeric character and underscore (A-z, 0-9, _).
- A variable name must start with a letter or underscore (_) character.
- A PHP variable name cannot contain spaces.
- One thing to be kept in mind that the variable name cannot start with a number or special symbols.
- PHP variables are case-sensitive, so $name and $NAME both are treated as different variable.

Example to Declare PHP Variables

- To store string, integer, and float values in PHP variables.

```
<?php
    $str="hello string";
    $x=200;
    $y=44.6;
    echo "string is: $str <br/>";
    echo "integer is: $x <br/>";
    echo "float is: $y <br/>";
?>
```

OUTPUT

string is: hello string

integer is: 200

float is: 44.6

Dr. Jyoti Rana, Naran Lala College, Navsari

Program 1

- Sum of two variables

```
<?php
    $x=5;
    $y=6;
    $z=$x+$y;
    echo $z;
?>
```

OUTPUT

11

Dr. Jyoti Rana, Naran Lala College, Navsari

PHP Variable: Case Sensitive

- In PHP, variable names are case sensitive. So variable name "color" is different from Color, COLOR, COLor etc.

```
<?php
    $color="red";
    echo "My car is " . $color . "<br>";
    echo "My house is " . $COLOR . "<br>";
    echo "My boat is " . $coLOR . "<br>";
?>
```

OUTPUT

My car is red

Notice: Undefined variable: COLOR in C:\wamp\www\variable.php on line 4

My house is

Notice: Undefined variable: coLOR in C:\wamp\www\variable.php on line 5

My boat is

Some Example of Variables

```
<?php
    $a="hello";//letter (valid)
    $_b="hello";//underscore (valid)
    echo "$a <br/> $_b";
    $4c="hello";//number (invalid)
    $*d="hello";//special symbol (invalid)
    echo "$4c <br/> $*d";
?>
```

OUTPUT

hello

Hello

Parse error: syntax error, unexpected '4' (T_LNUMBER), expecting variable (T_VARIABLE) or '$' in C:\wamp\www\variableinvalid.php on line 4

Note: PHP is a loosely typed language, it means PHP automatically converts the variable to its correct data type.

PHP Variable Scope

- The scope of a variable is defined as its range in the program under which it can be accessed. In other words, "The scope of a variable is the portion of the program within which it is defined and can be accessed."
- PHP has three types of variable scopes:
  - Local variable
  - Global variable
  - Static variable

LOCAL VARIABLE

- The variables that are declared within a function are called local variables for that function. These local variables have their scope only in that particular function in which they are declared. This means that these variables cannot be accessed outside the function, as they have local scope. A variable declaration outside the function with the same name is completely different from the variable declared inside the function.

```
<?php
    function local_var()
    {
        $num = 45;  //local variable
        echo "Local variable declared inside the function is: ". $num;
    }
    local_var();
?>
```

Output:

Local variable declared inside the function is: 45

Cont…

```
<?php
   function mytest()
   {
      $lang = "PHP";
      echo "Web development language: " .$lang;
   }
   mytest();
   //using $lang (local variable) outside the function will gener
   ate an error
   echo $lang;
?>
```

Output:

Web development language: PHP Notice: Undefined variable:
   lang in D:\xampp\htdocs\program\p3.php on line 28

GLOBAL VARIABLE

- The global variables are the variables that are declared outside the function. These variables can be accessed anywhere in the program. To access the global variable within a function, use the GLOBAL keyword before the variable. However, these variables can be directly accessed or used outside the function without any keyword. Therefore there is no need to use any keyword to access a global variable outside the function.

```
<?php
   $name = “Jyoti Rana";        //Global Variable
   function global_var()
   {
      global $name;
      echo "Variable inside the function: ". $name;
      echo "</br>";
   }
   global_var();
   echo "Variable outside the function: ". $name;
?>
```

OUTPUT

Variable inside the function: Jyoti Rana

Cont…

- Note: Without using the global keyword, if you try to access a global variable inside the function, it will generate an error that the variable is undefined.

```
<?php
    $name = "Sanaya Sharma";        //global variable
    function global_var()
    {
        echo "Variable inside the function: ". $name;
        echo "</br>";
    }
    global_var();
?>
```

Output:

Notice: Undefined variable: name in D:\xampp\htdocs\program\p3.php on line 6 Variable inside the function:

Using $GLOBALS instead of global

- Another way to use the global variable inside the function is predefined $GLOBALS array.

Example:

```
<?php
    $num1 = 5;      //global variable
    $num2 = 13;     //global variable
    function global_var()
    {
        $sum = $GLOBALS['num1'] + $GLOBALS['num2'];
        echo "Sum of global variables is: " .$sum;
    }
    global_var();
?>
```

Output:

Sum of global variables is: 18

Cont…

If two variables, local and global, have the same name, then the local variable has higher priority than the global variable inside the function.

```
<?php
    $x = 5;  //global variable
    function mytest()
    {
        $x = 7;  //local variable
        echo "value of x: " .$x;
    }
    mytest();
?>
```

Output:

Value of x: 7

Note: local variable has higher priority than the global variable.

Static variable

- It is a feature of PHP to delete the variable, once it completes its execution and memory is freed. Sometimes we need to store a variable even after completion of function execution. Therefore, another important feature of variable scoping is static variable. We use the static keyword before the variable to define a variable, and this variable is called as **static variable**.
- Static variables exist only in a local function, but it does not free its memory after the program execution leaves the scope.
- For Example,

Cont…

```
<?php
   function static_var()
   {
      static $num1 = 3;      //static variable
      $num2 = 6;         //Non-static variable
      //increment in static variable
      $num1++;
      //increment in non-static variable
      $num2++;
      echo "Static: " .$num1 ."</br>";
      echo "Non-static: " .$num2 ."</br>";
   }
//first function call
   static_var();

   //second function call
   static_var();
?>
```

Output:

Static: 4

Non-static: 7

Dr. Jyoti Rana, Naran Lala College, Navsari

Cont…

Note: You have to notice that $num1 regularly increments after each function call, whereas $num2 does not. This is why because $num2 is not a static variable, so it freed its memory after the execution of each function call.

PHP $ and $$ Variables

- The **$var** (single dollar) is a normal variable with the name var that stores any value like string, integer, float, etc.
- The **$$var** (double dollar) is a reference variable that stores the value of the $variable inside it.

```
<?php
$x = "abc";
$$x = 200;
echo $x."<br/>";
echo $$x."<br/>";
echo $abc;
?>
```

Output:

abc
200
200

Cont…

```
<?php
 $city="U.P";
$$city="Lucknow";
echo $city. "<br>";
echo $$city. "<br>";
echo "Capital of $city is " . $$city;
?>
```

OUTPUT:

U.P.

Lucknow

Capital of U.P. is Lucknow

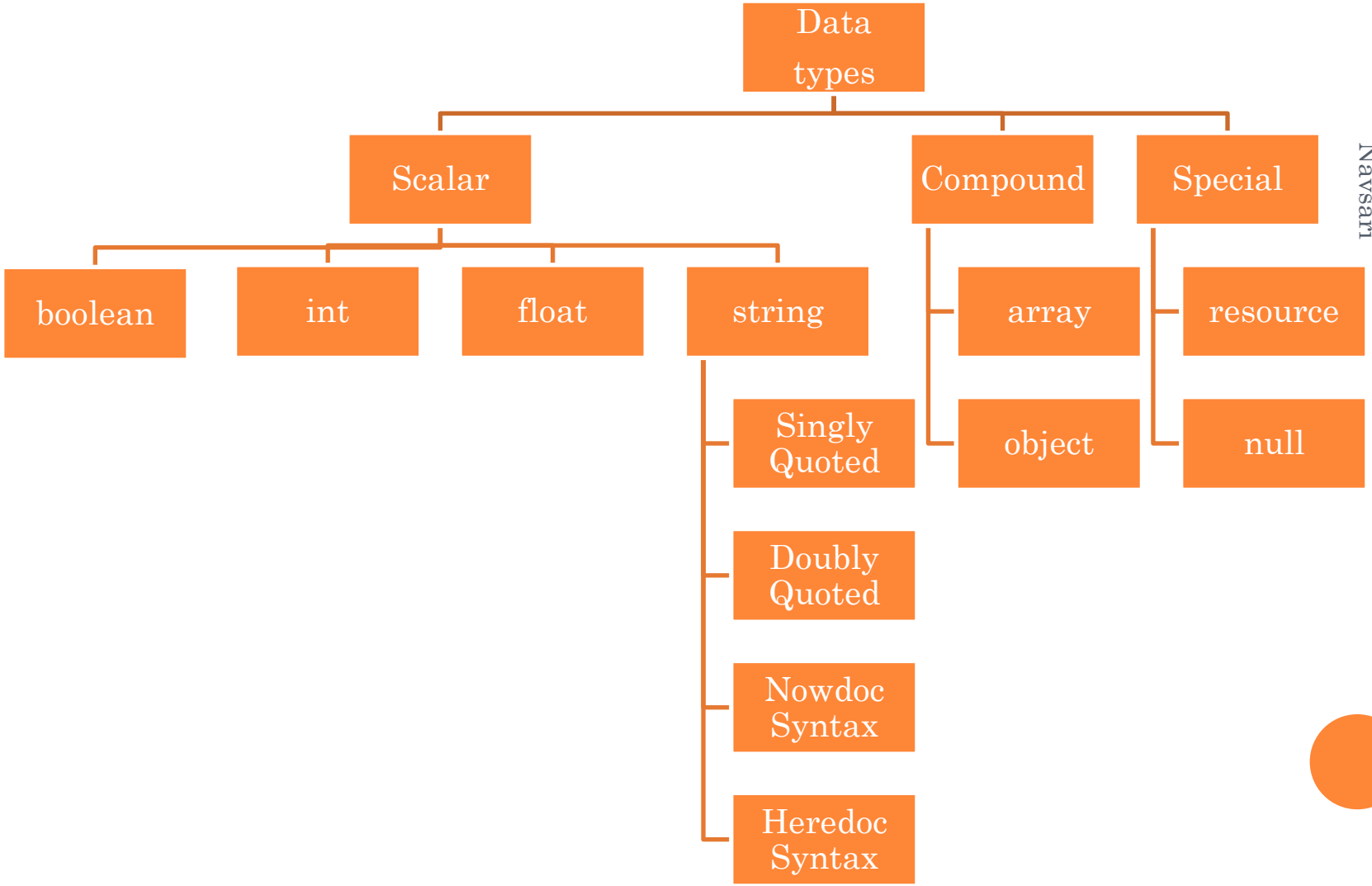
Constants in PHP

- A constant is an identifier (name) for a simple value. The value cannot be changed during the script.
- A valid constant name starts with a letter or underscore (no $ sign before the constant name).
- define() function is used to create a constant.
- define(*name*, *value[*, *case-insensitive]*)
- *case-insensitive*: Specifies whether the constant name should be case-insensitive. Default is false
- <?php
  define("GREETING", "Welcome to Naran Lala family!");
  echo GREETING;
  ?>

Difference between Echo and Print

Echo	Print
It is a language construct	It is a function
It does not return any value	It returns true on success and false otherwise
It accepts more than one value separated by commas	It accepts single value only
Echo works faster than print	Print works slower than echo
Echo “hello”, “TY”, “BCA”;	Print “hello”; Print “ty”; Print “bca”;

PHP Data Types



Dr. Jyoti Rana, Naran Lala College, Navsari

Scalar Data Type- Boolean

- Booleans are the simplest data type works like switch. It holds only two values: **TRUE (1)** or **FALSE (0)**. It is often used with conditional statements. If the condition is correct, it returns TRUE otherwise FALSE.
  - **Example:**

```
<?php
    if (TRUE)
        echo "This condition is TRUE.";
    if (FALSE)
        echo "This condition is FALSE.";
?>
```

Output:

This condition is TRUE.

Scalar Data Type- Integer

- Integer means numeric data with a negative or positive sign. It holds only whole numbers, i.e., numbers without fractional part or decimal points.
- **Rules for integer:**
  - An integer can be either positive or negative.
  - An integer must not contain decimal point.
  - Integer can be decimal (base 10), octal (base 8), or hexadecimal (base 16).
  - The range of an integer must be lie between 2,147,483,648 and 2,147,483,647 i.e., -2^31 to 2^31.
- **Example:**

```
<?php
    $dec1 = 34;
    $oct1 = 0243;
    $hexa1 = 0x45;
    echo "Decimal number: " .$dec1. "</br>";
    echo "Octal number: " .$oct1. "</br>";
    echo "HexaDecimal number: " .$hexa1. "</br>";
?>
```

Output:

Decimal number: 34 Octal number: 163 HexaDecimal number: 69

Scalar Data Type- Float

- A floating-point number is a number with a decimal point. Unlike integer, it can hold numbers with a fractional or decimal point, including a negative or positive sign.
- **Example:**

```
<?php
    $n1 = 19.34;
    $n2 = -54.472;
    $sum = $n1 + $n2;
    echo "Addition of floating numbers: " .$sum;
?>
```

Output:

Addition of floating numbers: 73.812

Dr. Jyoti Rana, Naran Lala College, Navsari

Scalar Data Type- String

- A string is a non-numeric data type. It holds letters or any alphabets, numbers, and even special characters.
- String values must be enclosed either within **single quotes** or in **double quotes**. But both are treated differently. To clarify this, see the example below:
  - **Example:**

```
<?php
    $company = "Javatpoint";
    //both single and double quote statements will treat different
    echo "Hello $company";
    echo "</br>";
    echo 'Hello $company';
?>
```

Output:

Hello Javatpoint
Hello $company

Dr. Jyoti Rana, Naran Lala College, Navsari

Singly Quoted String	Doubly Quoted String
It will be included in ‘	It will be included in “”
It supports only 2 escape sequences such as \\ - backslash \’ – single quote	It supports all escape sequences such as \n – new line \r – Carraige Return \\ - backslash \$ - Dollar Sign \” – Double Quotes \’ – Single Quotes \t – Tab Character
It doesn’t replace variable with it’s value	It replaces variable with it’s value
<?php echo 'This\'s a test'; $a =10; echo ' a is $a'; echo ' this is a Dev\'s Pen'; echo ' this course belongs to \'BCA\' ';	<?php echo "This's a test"; $a =10; echo " a is $a"; echo " this is a Dev's Pen"; echo " this course belongs to \"BCA\" ";

Cont…

- **Nowdoc syntax:** It is similar to singly quoted string. It is denoted by <<< sign and identifier is enclosed in single quote.
  - **Syntax :** <<< ‘identifier’ Ex. Echo <<<‘s1’

    …….. **This is example of nowdoc**

    ……. **Similar to single quote**

    Identifier; **s1;**
- **Heredoc syntax:** It is similar to doubly quoted string. It is denoted by <<< sign and then name of the identifier.
  - **Syntax :** <<< identifier Ex. Echo <<<s2

    …….. **This is example of heredoc**

    ……. **Similar to doublequote**

    Identifier; **s2;**

Dr. Jyoti Rana, Naran Lala College, Navsari

Compound Data Type- Array

- An array is a compound data type. It can store multiple values of same data type in a single variable.
- There are 3 types of array
  - Numeric Array
  - Associative Array
  - Multi-dimmensional Array
- **Example:**

```
<?php
    $bikes = array ("Royal Enfield", "Yamaha", "KTM");
    echo "Array Element1: $bikes[0] </br>";
    echo "Array Element2: $bikes[1] </br>";
    echo "Array Element3: $bikes[2] </br>";
?>
```

Output:

Array Element1: Royal Enfield
Array Element2: Yamaha
Array Element3: KTM

Compound Data Type- Object

- Objects are the instances of user-defined classes that can store both values and functions. They must be explicitly declared.
- **Example:**

```
<?php
    class bike {
        function model() {
            $model_name = "Royal Enfield";
            echo "Bike Model: " .$model_name;
        }
    }
    $obj = new bike();
    $obj -> model();
?>
```

Output:

Bike Model: Royal Enfield

Special Data Type- Resource

- Resources are not the exact data type in PHP. Basically, these are used to store some function calls or references to external PHP resources.
- **For example** - a database call, a file pointer
- It is an external resource.

Special Data Type- Null

- Null is a special data type that has only one value: **NULL**. There is a convention of writing it in capital letters as it is case sensitive.
- The special type of data type NULL defined a variable with no value.
- **Example:**

```
<?php
    $nl = NULL;
    echo $nl;   //it will not give any output
?>
```

Output:

Dr. Jyoti Rana, Naran Lala College, Navsari

PHP Operators

PHP Operator is a symbol i.e used to perform operations on operands. In simple words, operators are used to perform operations on variables or values.

- Arithmetic Operators
- Assignment Operators
- Bitwise Operators
- Comparison Operators
- Incrementing/Decrementing Operators
- Logical Operators
- String Operators
- Execution Operators
- Error Control Operators
- Array Operators
- Type Operators

Dr. Jyoti Rana, Naran Lala College, Navsari

Arithmetic Operators

- The PHP arithmetic operators are used to perform common arithmetic operations such as addition, subtraction, etc. with numeric values.

Operator	Name	Example	Explanation
+	Addition	$a + $b	Sum of operands
-	Subtraction	$a - $b	Difference of operands
*	Multiplication	$a * $b	Product of operands
/	Division	$a / $b	Quotient of operands
%	Modulus	$a % $b	Remainder of operands
**	Exponentiation	$a ** $b	$a raised to the power $b

Dr. Jyoti Rana, Naran Lala College, Navsari

Assignment Operators

- The assignment operators are used to assign value to different variables. The basic assignment operator is

Operator	Name	Example	Explanation
=	Assignment	$a = $b	The value of right operand is assigned to the left operand.
+=	Addition assignment	$a += $b	Addition same as $a = $a + $b
-=	Subtraction Assignment	$a -= $b	Subtraction same as $a = $a - $b
*=	Multiplication Assignment	$a *= $b	Multiplication same as $a = $a * $b
/=	Division assignment	$a /= $b	Find quotient same as $a = $a / $b
%=	Modulo Assignment	$a %= $b	Find remainder same as $a = $a % $b

Bitwise Operators

- The bitwise operators are used to perform bit-level operations on operands. These operators allow the evaluation and manipulation of specific bits within the

Operator	Name	Example	Explanation
&	And	$a & $b	Bits that are 1 in both $a and $b are set to 1, otherwise 0.
|	Or (Inclusive or)	$a | $b	Bits that are 1 in either $a or $b are set to 1
^	Xor (Exclusive or)	$a ^ $b	Bits that are 1 in either $a or $b are set to 0.
~	Not	~$a	Bits that are 1 set to 0 and bits that are 0 are set to 1
<<	Shift left	$a << $b	Left shift the bits of operand $a $b steps
>>	Shift right	$a >> $b	Right shift the bits of $a operand by $b number of places

Comparison Operators

- Comparison operators allow comparing two values, such as number or string

Operator	Name	Example	Explanation
==	Equal	$a == $b	Return TRUE if $a is equal to $b
===	Identical	$a === $b	Return TRUE if $a is equal to $b, and they are of same data type
!==	Not identical	$a !== $b	Return TRUE if $a is not equal to $b, and they are not of same data type
!=	Not equal	$a != $b	Return TRUE if $a is not equal to $b
<>	Not equal	$a <> $b	Return TRUE if $a is not equal to $b
<	Less than	$a < $b	Return TRUE if $a is less than $b
>	Greater than	$a > $b	Return TRUE if $a is greater than $b
<=	Less than or equal to	$a <= $b	Return TRUE if $a is less than or equal $b
>=	Greater than or equal to	$a >= $b	Return TRUE if $a is greater than or equal $b

Difference between =, == and ===

=	==	===
= is an “Assignment Operator”	== is an “equal to” operator	=== is an “identical equal to” operator
It is used to assign value to the variable.	It is used to compare values of 2 variables for equality.	It is used to compare data types and values of 2 variables.
--	It returns true if values of both variables are same.	It returns true if and only if values as well as data types of both the variables are same.
Example $a=10;	$a=10 ; $b=10.0; $f = ($a == $b); Echo $f; //true	$a=10 ; $b=10.0; $f = ($a === $b); Echo $f; //false

Incrementing/Decrementing Operators

- They are used to increase and decrease the value of a variable.

Operator	Name	Example	Explanation
++	Increment	++$a	Increment the value of $a by one, then return $a
		$a++	Return $a, then increment the value of $a by one
--	decrement	--$a	Decrement the value of $a by one, then return $a
		$a--	Return $a, then decrement the value of $a by one

Logical Operators

- They are used to perform bit-level operations on operands. These operators allow the evaluation and manipulation of specific bits within the integer.

Operator	Name	Example	Explanation
and	And	$a and $b	Return TRUE if both $a and $b are true
Or	Or	$a or $b	Return TRUE if either $a or $b is true
xor	Xor	$a xor $b	Return TRUE if either $ or $b is true but not both
!	Not	! $a	Return TRUE if $a is not true
&&	And	$a && $b	Return TRUE if either $a and $b are true
||	Or	$a || $b	Return TRUE if either $a or $b is true

String Operators

- They are used to perform the operation on strings.

Operator	Name	Example	Explanation
.	Concatenation	$a . $b	Concatenate both $a and $b
.=	Concatenation and Assignment	$a .= $b	First concatenate $a and $b, then assign the concatenated string to $a, e.g. $a = $a . $b

Execution Operators

- PHP has an execution operator **backticks (``)**. PHP executes the content of backticks as a shell command. Execution operator and **shell_exec()** give the same result.

Operator	Name	Example	Explanation
``	backticks	echo `dir`;	Execute the shell command and return the result. Here, it will show the directories available in current folder.

Error Control Operators

- PHP has one error control operator, i.e., **at (@) symbol**. Whenever it is used with an expression, any error message will be ignored that might be ge

Operator	Name	Example	Explanation
@	at	@file ('non_existent_file')	Intentional file error

Dr. Jyoti Rana, Naran Lala College, Navsari

PHP Operators Precedence

Operators	Additional Information	Associativity
[	array()	left
**	arithmetic	right
++ -- ~ (int) (float) (string) (array) (object) (bool) @	increment/decrement and types	right
instanceof	types	non-associative
!	logical (negation)	right
* / %	arithmetic	left
+ - .	arithmetic and string concatenation	left
<< >>	bitwise (shift)	left
< <= > >=	comparison	non-associative
== != === !== <>	comparison	non-associative

Conditional Statement
If

- PHP if else statement is used to test condition. There are various ways to use if statement in PHP.
  - if
  - if-else
  - if-else-if
  - nested if
- PHP If Statement
  - PHP if statement allows conditional execution of code. It is executed if condition is true.
  - If statement is used to executes the block of code exist inside the if statement only if the specified condition is true.

Syntax	Example

if(condition)
{
//code to be executed
}
	<?php
	$num=12;
	if($num<100)
	{
	echo "$num is less than 10
	0";
	}
	?>
	Output:

Dr. Jyoti Rana, Naran Lala College, Navsari

Conditional Statement
If-else

- PHP if-else statement is executed whether condition is true or false.
- If-else statement is slightly different from if statement. It executes one block of code if the specified condition is **true** and another block of code if the condition is **false**.

Syntax	Example

if(condition){
//code to be executed if tru
e
}**else**{
//code to be executed if fals
e
}
	<?php
	$num=12;
	if($num%2==0){
	echo "$num is even numbe
	r";
	}**else**{
	echo "$num is odd number
	";
	}
	?>
	Output:
	12 is even number

Dr. Jyoti Rana, Naran Lala College, Navsari

Conditional Statement
If-else-if

- The PHP if-else-if is a special statement used to combine multiple if?.else statements. So, we can check multiple conditions using this statement.

Syntax	Example

if (condition1){
//code to be executed if condition1 is true
} **elseif** (condition2){
//code to be executed if condition2 is true
} **elseif** (condition3){
//code to be executed if condition3 is true
....
} **else**{
//code to be executed if all given conditions are false
}
	<?php
	$marks=69;
	if ($marks<33){
	echo "fail";
	}
	else if ($marks>=34 && $marks<50) {
	echo "D grade";
	}
	else if ($marks>=50 && $marks<65) {
	echo "C grade";
	}
	else if ($marks>=65 && $marks<80) {
	echo "B grade";
	}
	else if ($marks>=80 && $marks<90) {
	echo "A grade";
	}
	else if ($marks>=90 && $marks<100) {
	echo "A+ grade";
	}
	else {
	echo "Invalid input";
	}
	?>
	Output:
	B Grade

Conditional Statement

nested if

- The nested if statement contains the if block inside another if block. The inner if statement executes only when specified condition in outer if statement is **true**.

Syntax	Example

if (condition) {
//code to be executed if condition is true
if (condition) {
//code to be executed if condition is true
}
}
	<?php
	$age = 23;
	$nationality = "Indian";
	if ($nationality == "Indian")
	{
	if ($age >= 18) {
	echo "Eligible to give vote";
	}
	else {
	echo "Not eligible to give vote";
	}
	}
	?>

Conditional Statement Switch

- PHP switch statement is used to execute one statement from multiple conditions. It works like PHP if-else-if statement.

Syntax	Example

switch(expression){
case value1:
 //code to be executed
 break;
case value2:
 //code to be executed
 break;
.....
default:
 code to be executed **if** all cases ar e not matched;
}
	<?php
	$num=20;
	switch($num){
	case 10:
	echo("number is equals to 10");
	break;
	case 20:
	echo("number is equal to 20");
	break;
	case 30:
	echo("number is equal to 30");
	break;
	default:
	echo("number is not equal to 10, 20 or 30");
	}
	?>
	Output:
	number is equal to 20

Conditional Statement Switch

- The **default** is an optional statement. Even it is not important, that default must always be the last statement.
- There can be only one **default** in a switch statement. More than one default may lead to a **Fatal** error.
- Each case can have a **break** statement, which is used to terminate the sequence of statement.
- The **break** statement is optional to use in switch. If break is not used, all the statements will execute after finding matched case value.
- PHP allows you to use number, character, string, as well as functions in switch expression.
- Nesting of switch statements is allowed, but it makes the program more complex and less readable.
- You can use semicolon (;) instead of colon (:). It will not generate any error.

To pass string in Switch expression

```
<?php
   $ch = “BCA";
   switch ($ch)
   {
      case "BCA":
         echo "BCA is 3 years course";
         break;
      case "Bsc":
         echo "Bsc is 3 years course";
         break;
      case "B.Tech":
         echo "B.Tech is 4 years course";
         break;
      case "B.Arch":
         echo "B.Arch is 5 years course";
         break;
      default:
         echo "Wrong Choice";
         break;
```

Case without Break

```
<?php
    $ch = 'c';
    switch ($ch)
    {
        case 'a':
            echo "Choice a";
            break;
        case 'b':
            echo "Choice b";
            break;
        case 'c':
            echo "Choice c";
            echo "</br>";
        case 'd':
            echo "Choice d";
            echo "</br>";
        default:
            echo "case a, b, c, and d is not found";
    }
```

Nested Switch

```
<?php
   $car = "Hyundai";
      $model = "Tucson";
      switch( $car )
      {
         case "Honda":
            switch( $model )
            {
               case "Amaze":
                  echo "Honda Amaze price is 5.9
3 - 9.79 Lakh.";
                  break;
               case "City":
                  echo "Honda City price is 9.91 -
 14.31 Lakh.";
                  break;
            }
            break;
         case "Renault":
            switch( $model )
            {
               case "Duster":
                  echo "Renault Duster price is 9.1
5 - 14.83 L.";
                  break;
               case "Kwid":
                  echo "Renault Kwid price is 3.1
5 - 5.44 L.";
                  break;
            }
            break;
         case "Hyundai":
            switch( $model )
            {
               case "Creta":
                  echo "Hyundai Creta price is 11.4
2 - 18.73 L.";
                  break;
      case "Tucson":
                  echo "Hyundai Tucson price is
22.39 - 32.07 L.";
                  break;
               case "Xcent":
                  echo "Hyundai Xcent price is 6.
5 - 10.05 L.";
                  break;
            }
            break;
      }
?>
```

PHP For Loop

- PHP for loop can be used to traverse set of code for the specified number of times.
- It should be used if the number of iterations is known otherwise use while loop. This means for loop is used when you already know how many times you want to execute a block of code.
- It allows users to put all the loop related statements in one place. See in the syntax given below:

Syntax

```
for(initialization; condition; increment/decrement){
//code t
}
```

Example 1	Example 2

<?php
for($n=1;$n<=10;$n++){
echo "$n
";
}
?>
	<?php
	$i = 1;
	//infinite loop
	for (;;) {
	echo $i++;
	echo "</br>";
	}
	?>

Dr. Jyoti Rana, Naran Lala College, Navsari

Nested For Loop

```
<?php
for($i=1;$i<=3;$i++){
for($j=1;$j<=3;$j++){
echo "$i   $j<br/>";
}
}
?>
```

1 1

1 2

1 3

2 1

2 2

2 3

3 1

3 2

3 3

Dr. Jyoti Rana, Naran Lala College, Navsari

PHP For Each Loop

PHP for each loop is used to traverse array elements. For every loop iteration, the values of the current array element is assigned to $value and array pointer is moved by one. So on the next loop iteration is

Syntax 1	Example 1

foreach($array **as** $value)
{
//code to be executed
}
?>
	<?php
	$season=**array**("summer","winter","spring",
	"autumn");
	foreach($season **as** $arr)
	{
	echo "Season is: $arr
	";
	}
	?>

Output:

Season is: summer

Season is: winter

Dr. Jyoti Rana, Naran Lala College, Navsari

PHP For Each Loop

Syntax 2	Example 2

foreach ($array **as** $key => $element)
{
//code to be executed
}
	<?php
	//declare array
	$employee = **array** (
	"Name" => “Simran",
	"Email" => “simmi@gmail.com",
	"Age" => 21,
	"Gender" => “Female"
	);
	
	//display associative array element through foreach loop
	foreach ($employee **as** $key => $element) {
	echo $key . " : " . $element;
	echo "</br>";
	}
	?>

PHP While Loop

- PHP while loop can be used to traverse set of code like for loop. The while loop executes a block of code repeatedly until the condition is FALSE. Once the condition gets FALSE, it exits from the body of loop.
- It should be used if the number of iterations is not known.
- The while loop is also called an **Entry control loop** because the condition is checked before entering the loop body. This means that first the condition is checked. If the condition is true, the block of code will be executed.

Syntax	Example

while(condition){
//code to be executed
}
	<?php
	$n=1;
	while($n<=10){
	echo "$n
	";
	$n++;
	}
	?>

while(condition):
//code to be executed
 	
endwhile;
	<?php
	$n=1;
	while($n<=10):
	echo "$n
	";
	$n++;
	endwhile;
	?>

PHP DO-WHILE LOOP

- PHP do-while loop can be used to traverse set of code like php while loop. The PHP do-while loop is guaranteed to run at least once.
- The PHP do-while loop is used to execute a set of code of the program several times. If you have to execute the loop at least once and the number of iterations is not even fixed, it is recommended to use the **do-while** loop.
- It executes the code at least one time always because the condition is checked after executing the code.

Syntax	Example

do{
//code to be executed
}**while**(condition);
	<?php
	$n=1;
	do{
	echo "$n
	";
	$n++;
	}**while**($n<=10);
	?>

Array

- An array is a special variable which can store multiple values in a single variable.
- For ex, to store names of 60 students of a class into single variable
- In PHP, there are 3 types of arrays:
  - Numeric Array
  - Associative Array
  - Multidimensional Array
- **Advantage of PHP Array**
  - **Less Code**: We don't need to define multiple variables.
  - **Easy to traverse**: By the help of single loop, we can traverse all the elements of an array.
  - **Sorting**: We can sort the elements of array.

Numeric Array

- A numeric array stores each array element with a numeric index.
  - By **assigning index manually**
    - $stud[1]="Anish";
    - $stud[2]="Bobby";
    - .
    - .
    - .
    - $stud[60]="Zenith";
  - By **using array() construct**
    - $gstud=array("Anish", "Bobby", "Zenith");

Cont…

- To access elements of numeric array
  - By specifying index only.
    - We can access particular element only.
    - For Example,
      - echo $stud[2];//Bobby
      - echo $stud[60];//Zenith
  - By displaying all the elements of an array
    - To display whole array, print_r() function is used.
    - For Example,
      - print_r($gstud);

Cont…

- By Assigning index manually

```
<?php
    $season[0]="summer";
    $season[1]="winter";
    $season[2]="spring";
    $season[3]="autumn";
    echo "Season are: $season[0], $season[1], $season[2] and $season
      [3]";
?>
```

- By using array() construct

```
<?php
    $season=array("summer","winter","spring","autumn");
    echo "Season are: $season[0], $season[1], $season[2] and $season
      [3]";
?>
```

- **Output:**

Season are: summer, winter, spring and autumn

Associative Array

- In this array, each element is associated with it's key. So, we don't want to remember index to access the element.
- We can associate name with each array elements in PHP using => symbol.
  - By **assigning index manually**
    - $age["Aditi"]=18;
    - $age["Dev"]=10;
    - .
    - .
    - .
    - $age["Jhanvi"]=14;
  - By **using array() construct**
    - $marks=array("Anjali"=>92,"Isha"=>95,"Maitry"=>55,"Yashavi"=>62);
    - $age = array("Aditi"=>18, "Dev"=>10, "Jhanvi"=>14);

Cont…

- To **access elements** of associative array
  - By **specifying index** only.
    - We can access particular element only.
    - For Example,
      - echo $age[“Jhanvi”];//14
      - echo $age[“Dev”];//10
  - By displaying all the elements of an array
    - To display whole array, **print_r()** function is used.
    - For Example,
      - print_r($marks);

Cont…

- **By Assigning index manually**

```
<?php
    $salary["Sonoo"]="350000";
    $salary["John"]="450000";
    $salary["Kartik"]="200000";
    echo "Sonoo salary: ".$salary["Sonoo"]."<br/>";
    echo "John salary: ".$salary["John"]."<br/>";
    echo "Kartik salary: ".$salary["Kartik"]."<br/>";
?>
```

- **By using array() construct**

```
<?php
    $salary=array("Sonoo"=>"350000","John"=>"450000","Kartik"=>
      "200000");
    echo "Sonoo salary: ".$salary["Sonoo"]."<br/>";
    echo "John salary: ".$salary["John"]."<br/>";
    echo "Kartik salary: ".$salary["Kartik"]."<br/>";
?>
```

Multidimensional Array

- PHP multidimensional array is also known as array of arrays. It allows you to store tabular data in an array. PHP multidimensional array can be represented in the form of matrix which is represented by row * column.
- Let's see a simple example of PHP multidimensional array to display following tabular data. In this example, we are displaying 3 rows and 3 columns.

Id	Name	Salary
1	Anjali	400000
2	Esha	500000
3	Yashvi	300000

- $emp = **array**(
  array(1,“Anjali",400000
  array(2,“Esha",500000)
  array(3,“Yashvi",30000
  );

Dr. Jyoti Rana, Naran Lala College, Navsari

Cont…

```
$families=array(
“shah”=>array(“Anish”,”Farhan”,”Yash”),
“patel”=>array(“Kalpesh”,”Mayur”,”hetav”),
“gandhi”=>array(“Jash”,”Swayam”)
);
// to display 1st “patel” students
   echo $families[“patel”][0];
// to display 3rd student from shah family
   echo $families[“shah”][2];
// to display whole array
   print_r($families);
// to display all students of gandhi family
   print_r($families[“gandhi”]);
```

Programs

1. Write a program to add elements of two 2*2 array.
2. Write a program to perform matrix addition, subtraction and multiplication.
3. Write a program

- To find max element
- To find min element
- To find average of all elements

from list of elements {50,95,35,20,40}.

Dr. Jyoti Rana, Naran Lala College, Navsari